

RISC201 ISA: Registers and Instruction Set

Register Organization

The processor uses 32 registers. Each register identifier is 5 bits wide because $2^5 = 32$. R0 is the ground register. R1 is used for the return address, R2 for the stack pointer, R3 for the frame pointer, and R4-R5 are reserved for kernel-level exception handling. R6-R30 are the 26 general-purpose registers.

Register	5-bit Code	Name / Role	Purpose
R0	00000	Ground Register	Always holds 0.
R1	00001	Return Address (RA)	Stores the return address for call/return operations.
R2	00010	Stack Pointer (SP)	Points to the current top of the stack.
R3	00011	Frame Pointer (FP)	Points to the current stack frame.
R4	00100	Kernel Register 0	Reserved for exception-handling operations.
R5	00101	Kernel Register 1	Reserved for exception-handling operations.
R6-R31	00110-11111	General-Purpose Registers	26 registers for normal program computation.

Register Details

R0: Ground Register: R0 always contains 0. Instructions using R0 as a source read 0. Writes to R0 do not change its value.

R1: Return Address (RA): R1 stores the address used to return from a procedure. The call instruction writes the return address into R1, and ret uses it to return.

R2: Stack Pointer (SP): R2 points to the current top of the stack. Stack operations and stack-frame management use R2.

R3: Frame Pointer (FP): R3 points to the current stack frame. It helps access local variables and function parameters using stable offsets.

R4: Kernel Register 0: R4 is reserved for kernel-level exception handling. Normal application code should not use it as a general-purpose register.

R5: Kernel Register 1: R5 is reserved for kernel-level exception handling and works with R4 when exception information needs to be preserved.

R6-R31: General-Purpose Registers: These 26 registers are available for normal arithmetic, logical, memory, shift, and other computations.

Instruction Encoding

Each instruction is encoded using 32 bits. The opcode field is 5 bits wide. The current ISA contains 25 instructions, so 5 opcode bits provide 32 possible opcode values.

Instruction	Opcode	Category	Brief Function
add	00001	Arithmetic	Adds two register values
sub	00010	Arithmetic	Subtracts one register value from another
mul	00011	Arithmetic	Multiplies two register values
div	00100	Arithmetic	Divides one register value by another
mod	00101	Arithmetic	Computes the remainder of division
cmp	00110	Comparison	Compares two values and updates the processor's comparison/condition information
and	00111	Logical	Performs bitwise AND
or	01000	Logical	Performs bitwise OR
not	01001	Logical	Performs bitwise NOT
xor	01010	Logical	Performs bitwise XOR
mov	01011	Data Transfer	Copies a value from one register to another
movu	01100	Immediate/Data	Moves an upper immediate value into the designated upper part of a register
movh	01101	Immediate/Data	Moves a high immediate value into the designated high part of a register
lsl	01110	Shift	Performs logical left shift
lsr	01111	Shift	Performs logical right shift
asr	10000	Shift	Performs arithmetic right shift while preserving the sign bit
nop	10001	Control	Performs no operation

ld	10010	Memory	Loads data from memory into a register
st	10011	Memory	Stores register data into memory
beq	10100	Branch	Branches when the comparison result indicates equality
bgt	10101	Branch	Branches when the comparison result indicates greater-than
bsm	10110	Branch	Branches when the comparison result indicates the first value is smaller than the second
b	10111	Branch	Unconditional branch to a target address
call	11000	Procedure	Calls a procedure and saves the return address in R1
ret	11001	Procedure	Returns from a procedure using the return address in R1

Instruction Explanations

add: Adds the values in two source registers and places the result in a destination register. It is used for integer addition.

sub: Subtracts the second operand from the first operand and stores the result in the destination register.

mul: Multiplies two source register values and stores the product in the destination register.

div: Divides one source register value by another and stores the quotient in the destination register. Division-by-zero handling will be defined by the exception framework.

mod: Computes the remainder after dividing one source value by another and stores the result in the destination register.

cmp: Compares two register values. The comparison result is used by conditional branch instructions such as beq, bgt, and branch-if-smaller.

and: Performs a bit-by-bit AND operation between two register values.

or: Performs a bit-by-bit OR operation between two register values.

not: Inverts every bit of the source register. A 0 becomes 1 and a 1 becomes 0.

xor: Performs bitwise exclusive OR. A result bit is 1 when the corresponding input bits differ.

mov: Copies the value of one register into another register.

movu: Loads an upper immediate value into the specified upper portion of a register. The exact immediate width and placement will be defined with the instruction format.

movh: Loads a high immediate value into the specified high portion of a register. The exact immediate width and placement will be defined with the instruction format.

lsl: Shifts all bits to the left. Zero bits enter from the right. Bits shifted out from the left are discarded.

lsr: Shifts all bits to the right. Zero bits enter from the left.

asr: Shifts bits to the right while copying the sign bit into the newly created positions. This preserves the signed nature of the value.

nop: Does nothing to the architectural state. It is useful for pipeline timing, spacing, or control purposes.

ld: Reads a value from memory and places it into a register. The exact address calculation will be defined by the instruction format.

st: Writes the value of a register to memory. The exact address calculation will be defined by the instruction format.

beq: Performs a conditional branch when the comparison condition represents equality.

bgt: Performs a conditional branch when the comparison condition represents greater-than.

bsm: Performs a conditional branch when the comparison condition indicates that the first compared value is smaller than the second.

b: Performs an unconditional branch. Program control moves directly to the specified target.

call: Transfers control to a procedure and saves the return address in R1.

ret: Transfers control back to the address stored in R1.

-----X-----X-----

4-Stage RISC201 Processor

The 4-stage design combines some operations to keep the pipeline short.

IF: The PC selects the next instruction. The instruction memory supplies the 32-bit instruction. The PC normally advances to the next instruction.

ID: The opcode is decoded. Register fields are used to read the register file. Immediate fields are prepared. Control signals for the later stage are generated.

EX/MEM: The ALU performs arithmetic, logical, and shift operations. For ld and st, the ALU calculates the effective memory address. For conditional branches, the comparison condition is evaluated and the target address is selected. Memory access occurs in this stage.

WB: Instructions producing a register result write it into the destination register. Store and branch instructions do not write a normal destination register.

6-Stage RISC201 Processor

The 6-stage design separates operations more finely. This gives shorter work per stage and a higher potential clock frequency, while introducing more pipeline stages and more opportunities for pipeline hazards.

IF: The PC selects and fetches the next 32-bit instruction.

ID: The instruction opcode and fields are decoded. The processor determines which execution path is required.

RR: The register file supplies source operands. Immediate values are extended and prepared for execution.

EX: The ALU performs arithmetic, logical, and shift operations. Effective addresses are calculated. Branch conditions are evaluated.

MEM: Load instructions read data memory. Store instructions write data memory. Non-memory instructions pass through without a data-memory operation.

WB: The result is written to the destination register for instructions requiring register write-back.

4-Stage vs 6-Stage Processor Behavior

	4-Stage	6-Stage
Work per stage	More work per stage because EX and memory access are combined.	Less work per stage because register read and memory access have separate stages.
Potential clock frequency	Lower, since each stage carries more logic.	Higher, since each stage carries less logic.
Pipeline complexity	Lower	Higher
Data hazards	Fewer stage boundaries, simpler hazard paths.	More pipeline boundaries, so forwarding and stalls need more careful control.
Control hazards	Branch decision occurs later than IF but earlier than WB.	Branch decision occurs in EX after more instructions have entered the pipeline.
Load operation	Address calculation and memory access share EX/MEM.	Address calculation occurs in EX and memory access occurs in MEM.
Implementation effort	Simpler CPU and control logic.	More pipeline registers and more control logic.
Ideal pipeline throughput	Up to one completed instruction per cycle after filling, subject to hazards.	Up to one completed instruction per cycle after filling, subject to hazards.
Single-instruction latency	Fewer pipeline stages.	More pipeline stages.

-----X-----X-----

Microprogrammed Control Unit for RISC201

The RISC201 assignment requires a corresponding microprogrammed control unit, with the choice of horizontal or vertical control. For our design, we use a horizontal microprogrammed control unit. The same ISA and register organization are used by both the 4-stage and 6-stage processors. The microprograms differ in the number of execution stages they control.

Why Horizontal Microprogramming

- Each microinstruction directly controls datapath operations through individual control bits.
- It fits the relatively large set of datapath actions needed by arithmetic, logical, memory, branch, call, and return instructions.
- It makes the control signals easy to connect to the processor datapath.
- It gives a simple step-by-step microPC view for the simulator required by the assignment.

Microprogrammed Control Unit Block Structure

The control unit contains the following major components:

Component	Function	Input / Output
Instruction Register (IR)	Holds the current 32-bit instruction.	Opcode and operand fields
Opcode Decoder / Dispatch Logic	Maps the 5-bit opcode to the starting microprogram address for the instruction.	Opcode -> microPC start address
Microprogram Counter (microPC)	Holds the address of the current microinstruction.	Control Memory address
Control Memory	Stores the horizontal microinstructions.	microPC -> control word

Microinstruction Register (MIR)	Holds the current control word while it drives the datapath.	Control bits
Next-Address / Sequencing Logic	Selects the next microinstruction.	Sequential, dispatch, branch, or condition
Condition / Flag Inputs	Provides comparison, zero, sign, overflow, memory, and exception conditions when required.	Status -> sequencing decision

Horizontal Microinstruction Format

A horizontal microinstruction uses one control bit for each directly controlled datapath action. The exact physical width is an implementation choice. For the first design, we use the following logical control fields.

Field	Width	Purpose	Example Signals
PC Control	4 bits	Controls PC update and selection.	PC_INC, PC_LOAD, PC_SRC
IR Control	1 bit	Loads the fetched instruction into IR.	IR_LOAD
Register File	5 bits	Controls register read/write actions.	REG_WRITE, source read enables
ALU Control	5 bits	Selects the ALU operation.	ADD, SUB, AND, OR, XOR, MUL, DIV, MOD
Shift Control	3 bits	Selects shift operation.	LSL, LSR, ASR
Memory Control	2 bits	Controls data-memory read/write.	MEM_READ, MEM_WRITE
Address / Operand Mux	4 bits	Selects sources for ALU/address inputs.	REG, IMM, PC, SP, FP
Branch Control	4 bits	Controls conditional/unconditional branch decisions.	BEQ, BGT, BSM, B
Exception Control	2 bits	Controls exception entry/handling actions.	EXC_SAVE, EXC_VECTOR
Microsequence Control	4 bits	Controls microprogram sequencing.	NEXT, DISPATCH, COND, END

The listed widths describe the logical control organization rather than a final hardware bit count. Some controls, such as register read selection, need the 5-bit register identifiers already present in the instruction. The final control-word width should be fixed after the datapath and exact instruction formats are finalized.

Common 4-Stage Microprogram

The 4-stage processor uses the following microcoded stages. A single microinstruction controls the main actions of each stage.

microPC	Stage	Main Controls	Operation	Next
00	IF	MEM instruction read, IR_LOAD, PC_INC	Fetch instruction and advance PC.	01
01	ID	Opcode dispatch, register read, immediate prepare	Decode instruction and prepare operands.	Instruction entry
02	EX/MEM	ALU / shift / branch / memory controls	Execute operation, calculate address, evaluate branch, or access data memory.	03
03	WB	REG_WRITE where required	Write result to destination register.	00

Common 6-Stage Microprogram

The 6-stage processor separates register read and memory access into their own pipeline stages. The microprogram therefore has six main stage-control points.

microPC	Stage	Main Controls	Operation	Next
00	IF	Instruction memory read, IR_LOAD, PC_INC	Fetch instruction and advance PC.	01
01	ID	Opcode decode, control dispatch	Decode the instruction.	02
02	RR	Register file read, immediate preparation	Read source registers and prepare immediate operands.	03
03	EX	ALU / shift / branch / address controls	Perform computation, evaluate branch, or calculate memory address.	04
04	MEM	MEM_READ / MEM_WRITE	Read or write data memory. Non-memory operations pass through.	05
05	WB	REG_WRITE where required	Write result to destination register.	00

Control Signals

The following signals form the main horizontal control set for the datapath. A signal is asserted in a microinstruction only when its operation is needed.

Signal	Meaning	Typical Use
IR_LOAD	Load instruction register	IF
PC_INC	Increment PC	IF
PC_LOAD	Load a new PC value	Branches, call, ret
PC_SRC	Select source of new PC	PC, branch target, R1
REG_READ	Enable register-file read	RR / ID
REG_WRITE	Write destination register	WB
ALU_ADD / SUB / MUL / DIV / MOD	Select arithmetic operation	Arithmetic instructions
ALU_AND / OR / XOR / NOT	Select logical operation	Logical instructions
SHIFT_LSL / SHIFT_LSR / SHIFT_ASR	Select shift operation	Shift instructions
MEM_READ	Read data memory	ld
MEM_WRITE	Write data memory	st
OP_A_SEL / OP_B_SEL	Select ALU input sources	Register, immediate, PC, SP, FP
CMP_UPDATE	Update comparison information	cmp
BR_COND	Select branch condition	beq, bgt, bsm
EXC_SAVE	Save exception information	Exception entry
EXC_VECTOR	Select exception vector	Exception handling

4-Stage vs 6-Stage Microprogrammed Control

Property	4-Stage Control	6-Stage Control
Main stage sequence	IF -> ID -> EX/MEM -> WB	IF -> ID -> RR -> EX -> MEM -> WB
Microprogram length for normal instruction	4 main stage microinstructions	6 main stage microinstructions
Register read	Combined with decode	Separate RR stage
Memory access	Combined with EX/MEM	Separate MEM stage
Control complexity	Lower	Higher
Pipeline timing control	Simpler	More detailed
Hazard support	Forwarding and stalls across fewer stages	Forwarding and stalls across more stage boundaries

Exceptions and Kernel Registers

R4 and R5 are reserved for kernel-level exception handling. The microprogrammed control unit provides exception control signals so an exception routine can preserve required processor information before transferring control to the exception handler. The exact exception vector and saved-state layout should be fixed together with the stack-protection and runtime exception design.

-----X-----X-----

RISC201 ALU Design

The RISC201 processor requires a corresponding ALU with arithmetic and logical operations. The ALU designed here is a 32-bit ALU and supports the operations required by the current RISC201 instruction set. The same ALU datapath is used by both the 4-stage and 6-stage processors. The difference between the processors is the pipeline organization, not the programmer-visible ALU operations.

ALU Operations

Operation	Instruction(s)	Function	Main Hardware
ADD	add	$A + B$	32-bit carry-lookahead adder
SUB	sub	$A - B$	Adder with B inversion and carry-in = 1
MUL	mul	$A \times B$	Signed 32-bit Booth multiplier
DIV	div	A / B	Signed division unit
MOD	mod	$A \bmod B$	Signed division unit, remainder output
CMP	cmp	Compare A and B	Subtraction and condition generation
AND	and	$A \& B$	Bitwise AND
OR	or	$A \mid B$	Bitwise OR
XOR	xor	$A \wedge B$	Bitwise XOR
NOT	not	$\sim A$	Bitwise inverter
LSL	lsl	$A \ll x$	Left-shift network
LSR	lsr	$A \gg x$	Logical right-shift network
ASR	asr	Signed $A \gg x$	Arithmetic right-shift network

MOV	mov/ movu/ movh	$rd \leftarrow rs1/imm$	Move contents of one register/Immediate to another
-----	-----------------	-------------------------	--

Arithmetic Unit

Addition

The ALU uses a 32-bit carry-lookahead adder for addition. Carry-lookahead reduces the dependency on a long ripple-carry chain by computing carry information using generate and propagate signals.

For each bit, generate and propagate terms are formed. The carry equations then determine the carry entering each position. The final sum bits are produced from the operands and their corresponding carries.

Addition operation is also used for instructions store and load.

Subtraction

Subtraction uses the same adder hardware. $A - B$ is computed as $A + (\sim B) + 1$. The ALU therefore complements B and supplies 1 as the initial carry input. This avoids a separate subtractor.

Comparison

cmp uses the subtraction path. The ALU computes $A - B$ and generates comparison information from the result and sign/overflow information. The comparison information is then used by beq, bgt, and bsm.

Condition	Meaning	Use
Zero = 1	A and B are equal	beq
Signed greater-than	$A > B$	bgt
Signed smaller-than	$A < B$	bsm

Multiplication

The multiplication unit uses a signed Booth multiplier. Booth encoding reduces the number of required addition/subtraction operations when the multiplier contains consecutive runs of 1s. The 32-bit operands produce a 64-bit internal product. The ISA result width is 32 bits, so the processor must define how the 64-bit product is reduced to the architectural 32-bit result.

Division and Modulo

The division unit computes both quotient and remainder so div and mod use the same hardware. A restoring or non-restoring signed division algorithm is suitable for the first implementation. The quotient is returned for div, while the remainder is returned for mod. Division by zero raises a runtime exception instead of producing a normal ALU result.

Logical Unit

Operation	Bit-level rule	Instruction
AND	Result bit = 1 only when both input bits are 1	and
OR	Result bit = 1 when either input bit is 1	or
XOR	Result bit = 1 when the input bits differ	xor
NOT	Each input bit is inverted	not

Shift Unit

The shift unit operates on a 32-bit operand. A shift amount is supplied by the instruction format or register operand definition selected for the ISA.

Operation	Direction	Bits entering	Instruction
Logical left shift	Left	Zeros enter from the right	lsl
Logical right shift	Right	Zeros enter from the left	lsr
Arithmetic right shift	Right	Sign bit is copied from the left	asr

Move Unit

MOV copies the value of the source register to the destination register. The ALU does not perform an arithmetic or logical operation. It selects the source operand A directly and forwards it to the ALU result.

Interaction with the RISC201 Instructions

Instruction	ALU Role	Destination / Use
add	$A + B$	Register result
sub	$A - B$	Register result
mul	$A \times B$	Register result
div	A / B	Register quotient
mod	$A \bmod B$	Register remainder
cmp	$A - B$ and condition generation	Branch condition information
and / or / xor	Bitwise logical operation	Register result
mov/ movh/ movu	Data Transfer	Register result
not	Bitwise inversion	Register result
lsl / lsr / asr	Shift operation	Register result
ld	Effective address = base + offset	Memory address
st	Effective address = base + offset	Memory address
beq / bgt / bsm	Comparison/condition evaluation	PC control
b	Target-address calculation if required	PC control
call	Return-address and target calculations as required by datapath	PC / R1
ret	PC target from R1	PC control

ALU in the 4-Stage Processor

In the 4-stage processor, the ALU operates in the EX/MEM stage. Arithmetic, logical, shift, comparison, effective-address, and branch calculations occur in this stage. For ld and st, the ALU calculates the effective address before the memory operation.

ALU in the 6-Stage Processor

In the 6-stage processor, the ALU operates in the EX stage. Register operands have already been read in the RR stage. The resulting value then passes through the MEM stage before register write-back for normal register-producing instructions.

Exceptions Related to the ALU

- Division by zero is detected by the division unit and sent to the runtime exception mechanism.
- Signed arithmetic overflow is indicated by the overflow status signal.
- The final exception policy, including which exceptions trap and what information is saved in R4 and R5, should be fixed with the processor exception framework.